

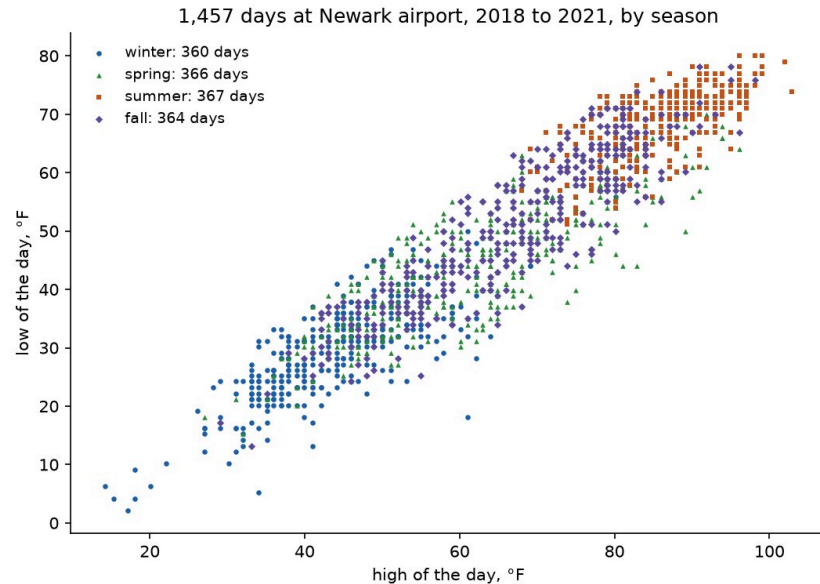
A Layer of Neurons Is a Matrix

Four neurons read a day of weather and name its season; five read a pixel and name its colour. Each layer is one matrix, and applying it to the data is one product.

The data: days at Newark airport

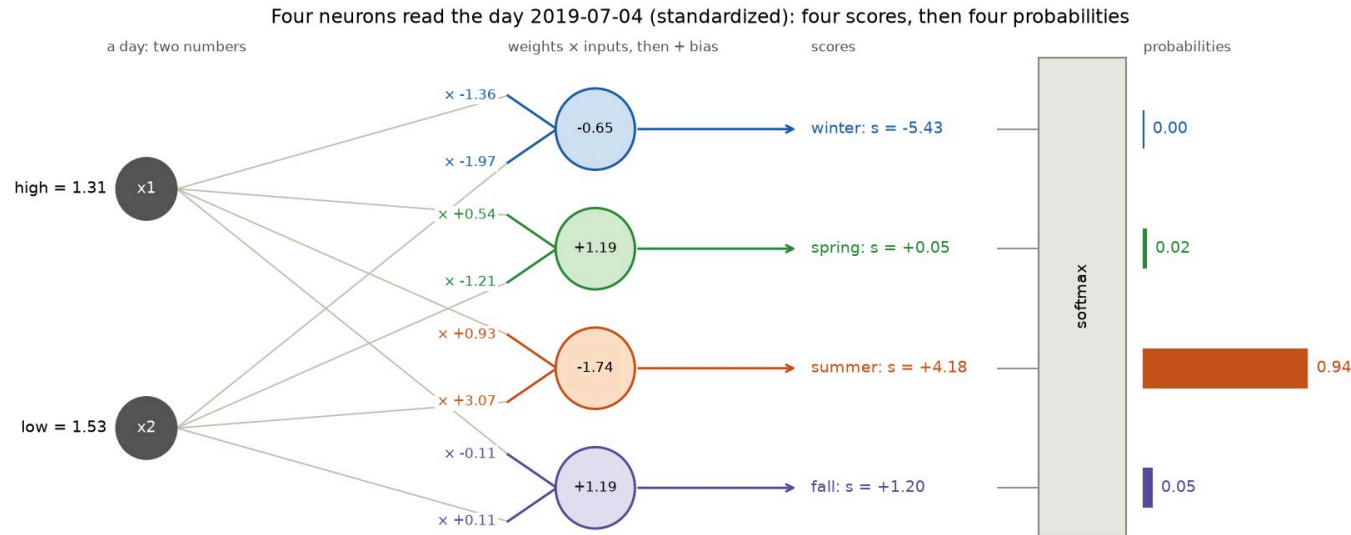
The days of 2018 to 2021 are $n = 1,457$ points with $m = 2$ coordinates each. The first four:

date	tmax_f	tmin_f
2018-01-01	20.1	6.3
2018-01-02	27.1	12.2
2018-01-03	30.2	10.2



- Newark airport, 2006 to 2024: the high and the low of each day
- One **point** = one day, $m = 2$ coordinates
- $n = 1,457$ days of 2018 to 2021: the rows of one matrix
- Standardized, as in meeting 10
- **Label**: the season, from the month

A neuron: a rule that reads a day



- One **weight** for each coordinate, and one **bias**
- **Score** = the weighted sum of the coordinates, plus the bias
- Four neurons read the same two numbers; each has its colour
- Softmax: four scores become four probabilities
- Weights found by **training** on 2006 to 2017

One neuron, one day: the score by hand

$$s = w_1x_1 + w_2x_2 + b$$

Day 2019-07-04: high 89 °F, low 74 °F. Standardized: $z = (1.31, 1.53)$.

The winter neuron: $w = (-1.36, -1.97)$, $b = -0.65$.

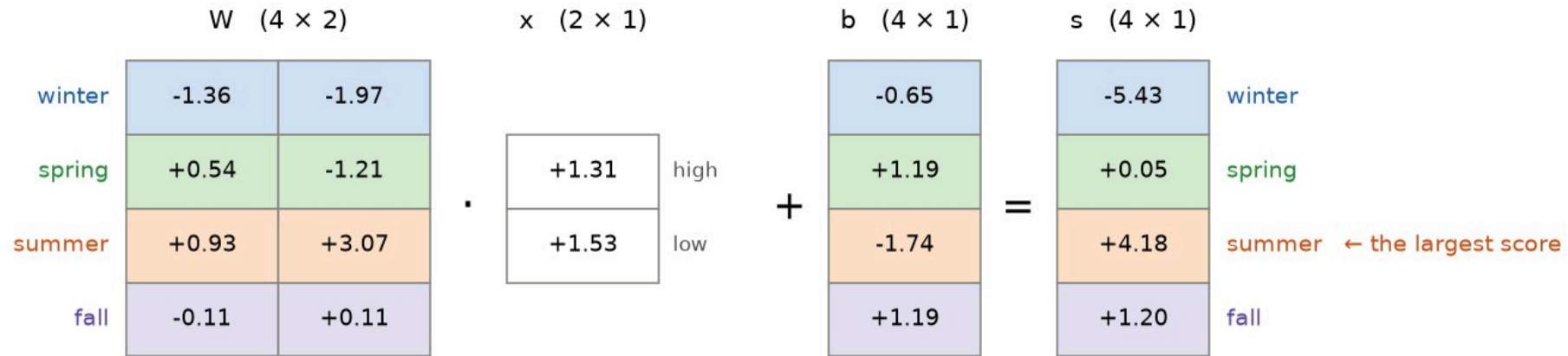
Its score: $-1.36 \times 1.31 + (-1.97) \times 1.53 + (-0.65) = -5.43$

shape of w : $(2,)$, shape of z : $(2,)$; $w @ z + b$ is one number.

- A large negative score: the neuron does not **fire**
- As matrices: $s = w^T x + b$, with w and x columns
- The **transpose** writes w as a row; a row times a column is one number
- In NumPy: shape $(2,)$, and `w @ x + b`

Four neurons, one day: the weights are a matrix

$$s = Wx + b$$



- **Row j of W : the weights of neuron j** , in its colour
- W : 4×2 . x : 2×1 . Wx : 4×1 , and b too
- $W @ x + b$: four scores in one line. The largest: the **argmax**

From scores to probabilities: the softmax

$$p_k = \frac{e^{s_k}}{e^{s_1} + e^{s_2} + \dots + e^{s_K}}$$

```
def softmax_rows(S):  
    shifted = S - S.max(axis=1, keepdims=True)    # the largest entry of every row becomes 0  
    E = np.exp(shifted)  
    return E / E.sum(axis=1, keepdims=True)
```

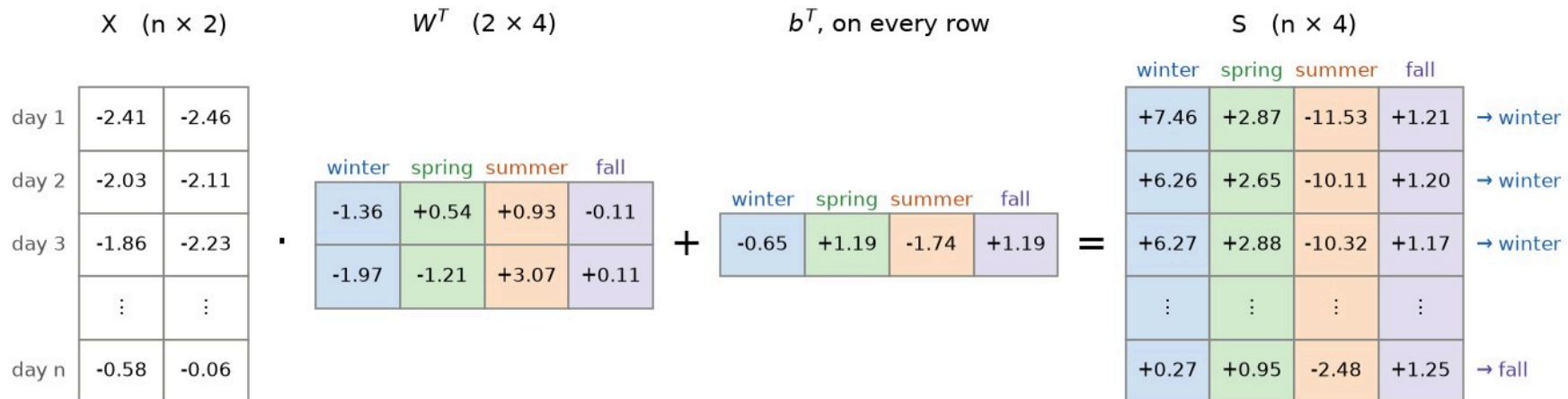
the four scores:	winter -5.43, spring +0.05, summer +4.18, fall +1.20	
their probabilities:	winter 0.000, spring 0.015, summer 0.937, fall 0.048	(they add to 1.000)
sigmoid of each score:	winter 0.004, spring 0.513, summer 0.985, fall 0.769	(each entry by itself)

- K scores become K positive numbers that add to 1
- A function of the **row**: every probability depends on all the scores
- The **sigmoid**, $\sigma(s) = 1 / (1 + e^{-s})$: a function of each **entry**, from 0 to 1
- Its numbers do not add to 1

Four neurons, many days: one product

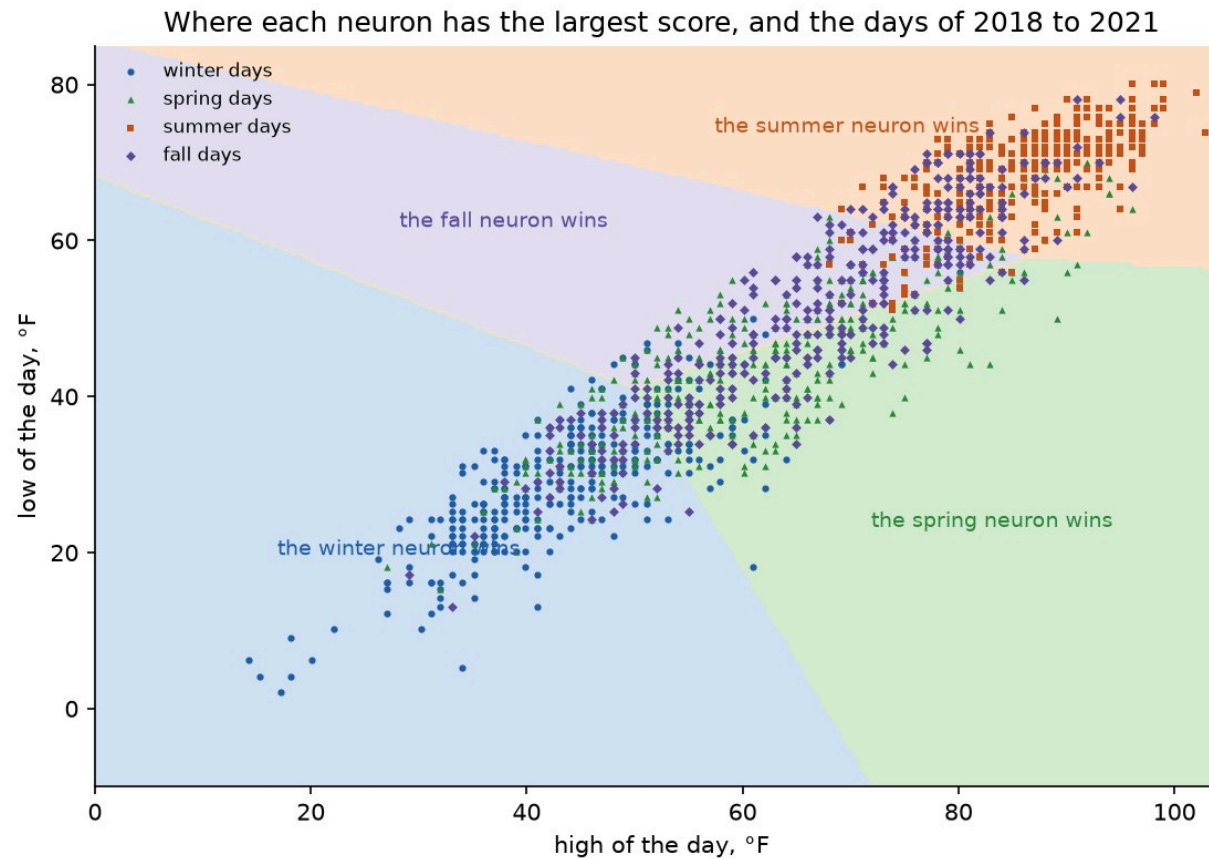
`S_days = Z_days @ W_seasons.T + b_seasons` *# the line of Step 4*

$$S = XW^T + 1b^T$$



- Columns of W^T and of S : one neuron each, in its colour. Rows of S : the days
- b^T on every row: **broadcasting**. W transposed: the neurons become columns

Where each neuron wins



- Coloured by the neuron with the largest score; the days in the colour of their true season
- 59.3% of the days right; spring and fall overlap

The four cases

	one point x , shape $(m,)$	n points X , shape (n, m)
one neuron: w of shape $(m,)$, b a number	$w @ x + b$ is a number	$X @ w + b$ has shape $(n,)$; b is broadcast
K neurons: W of shape (K, m) , b of shape $(K,)$	$W @ x + b$ has shape $(K,)$	$X @ W.T + b$ has shape (n, K) ; b is broadcast

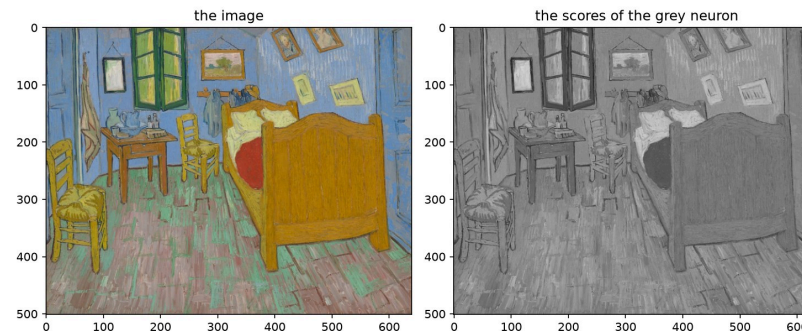
- The bias needs help only when there are many points
- $K = m$: $X @ W + b$ runs, and its numbers are wrong
- The same four lines on any data. Next: the pixels of your image

A second example: a neuron that scans an image

```
X = image.reshape(-1, 3)           # one row for each pixel
n, m = X.shape
print(f"shape of X: {X.shape}, which is n = {n:,} points with m = {m} coordinates")

s_grey = X @ w_grey + b_grey        # the scores of the grey neuron on all the points
print("shape of X @ w_grey + b_grey:", s_grey.shape)
```

shape of X: (320640, 3), which is n = 320,640 points with m = 3 coordinates
shape of X @ w_grey + b_grey: (320640,)

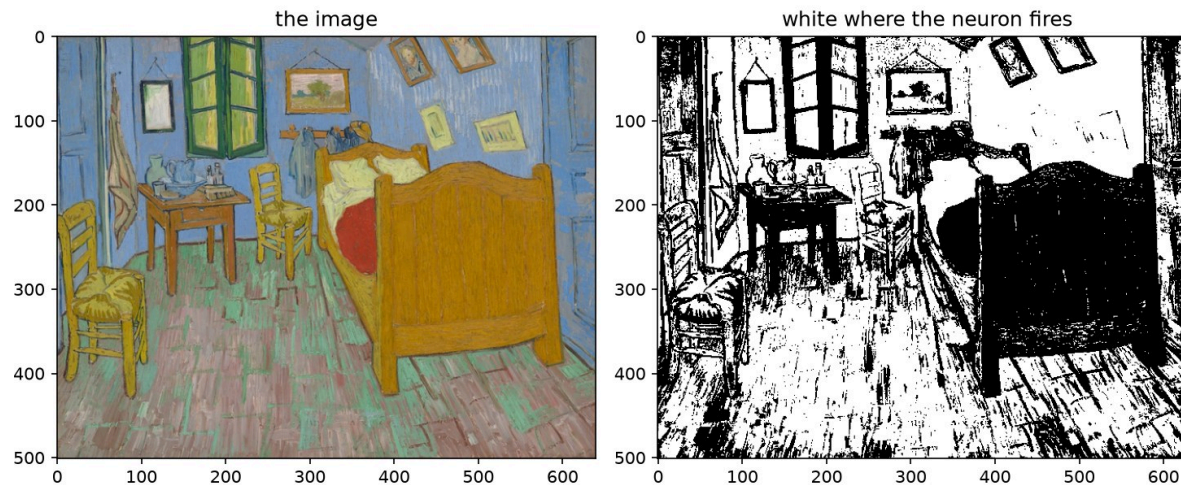


- A point = one **pixel**, m = 3: red, green, blue. `image.reshape(-1, 3)`: n = $H \times W$ rows
- Weights 0.299, 0.587, 0.114, bias 0: the score is the brightness. The scores, drawn: the grey image

A neuron that fires

```
b_bright = -s_grey.mean() # minus the mean brightness of the image
fires = X @ w_grey + b_bright >= 0 # True for the points on which the neuron fires
print(f"bias {b_bright:.3f}: the neuron fires on {fires.mean():.1%} of the {n:,} pixels")
```

bias -0.518: the neuron fires on 53.1% of the 320,640 pixels



- Bias 0: a brightness is never negative, so the neuron fires on every pixel
- Bias $-(\text{mean brightness})$: fires on the pixels brighter than the average. $\theta = -b$ is the **threshold**

Five neurons, one for each reference colour

```
K = 5
```

```
C = reference_colours(X, K)
```

```
W = C.copy()
```

```
b = -0.5 * (C ** 2).sum(axis=1)
```

K rows and 3 columns: the reference colours, darkest first

the weights of the layer: row j is reference colour j

the biases of the layer: one number for each neuron

W (5 × 3): row j, in its own colour

	red	green	blue
neuron 0	0.32	0.36	0.31
neuron 1	0.63	0.46	0.10
neuron 2	0.53	0.53	0.45
neuron 3	0.48	0.57	0.66
neuron 4	0.67	0.65	0.55

- Five **reference colours** of the image, found by k-means
- Row j of W: the colour c_j itself. Bias: $-\frac{1}{2}|c_j|^2$
- **Largest score = nearest colour**
- As the seasons named a day, the colours name a pixel

The scores of one pixel

$$s_x = W @ x + b$$

the K scores of the pixel x

the pixel $x = [0.56 \ 0.39 \ 0.1 \]$

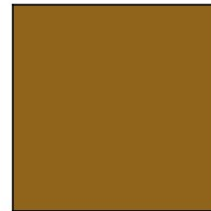
shape of $W @ x + b$: (5,)

the scores of the 5 neurons: $[0.19 \ 0.237 \ 0.17 \ 0.065 \ 0.104]$

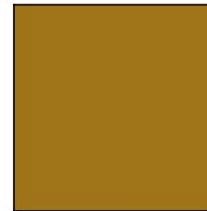
The largest score is that of neuron 1, whose colour is $[0.63 \ 0.46 \ 0.1 \]$.

The pixel, and the colour with the largest score

the pixel x



colour of neuron 1



- $W @ x + b$, as for the four neurons and one day
- Neurons are numbered from 0

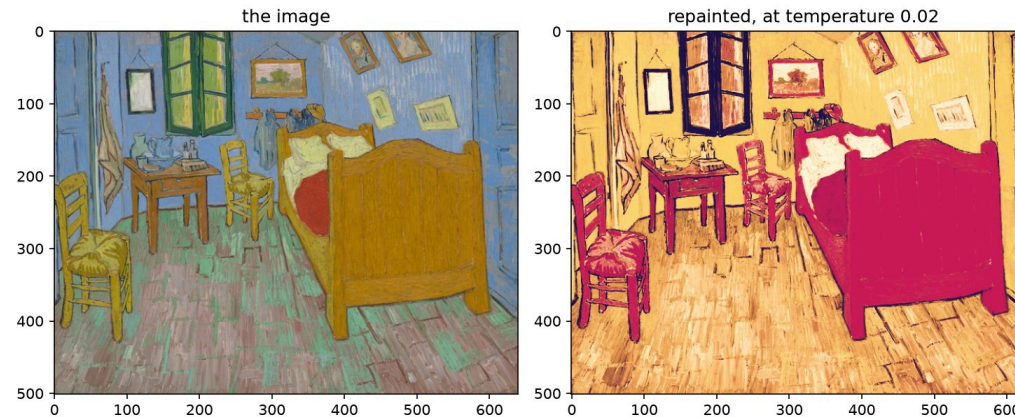
Many neurons, many points: the three loops

```
def layer_loops(X, W, b):  
    n, m = X.shape  
    K = W.shape[0]  
    S = np.empty((n, K))  
    for i in range(n):                # every point  
        for j in range(K):            # every neuron  
            total = b[j]  
            for c in range(m):         # every coordinate  
                total += W[j, c] * X[i, c]  
            S[i, j] = total  
    return S
```

- S: $n \times K$. Row i: pixel i. Column j: neuron j
- Every point, every neuron, every coordinate
- For the days this was one line. **In Colab you write that line for the pixels**

Repaint the image: a small network

```
Prob = softmax_rows(layer(X, W, b) / T)    # n rows and K columns  
new = Prob @ PAINT                        # n rows and 3 columns
```



- A **paint** for each neuron: the rows of `PAINT`, $K \times 3$
- `Prob @ PAINT`: $n \times 3$, each pixel a mixture of the paints
- A second product of the same kind: n points, K coordinates, three neurons
- Two layers with a function between them: a small **neural network**

Now in Colab: Act 1

- The class notebook: GPU asked for, setup run, your number typed
- **Act 1:** run the cell that builds the layer of five neurons on your image
- **Write the one line** of `layer`: the loops as one product, as for the days. The check tells you what to transpose
- Repaint: five paints of your choice; temperatures 1, 0.1, 0.002
- **Post:** the printed line, your image beside the repainted one with one sentence, what the temperature changed
- One member posts; the names of all the members first

Loops against the product

`n = 320,640 points, m = 3 coordinates, K = 5 neurons`

`The innermost line of the loops runs  $n \times K \times m = 4,809,600$  times.`

version	seconds
---------	---------

three loops	1.6179
-------------	--------

one product	0.0106
-------------	--------

`The loops take 152 times as long as the product.`

- The innermost line runs $n \times K \times m$ times
- The product: the same arithmetic, inside NumPy
- Meeting 12 counts the operations. Act 2: the same line in PyTorch, and on a GPU